

EXPERIMENT-6

1. Imagine you are developing an Employee Management System, where employee details need to be maintained securely. Create a package named `EmployeeManagement` that contains a class `Employee` with attributes such as name, id, and salary. The class should also have a method `displayDetails()` that prints employee information. Then, create another program outside the `EmployeeManagement` package that imports the package and utilizes the `Employee` class to create an employee object and display their details. Ensure that the package maintains encapsulation while allowing controlled access to employee information.

```
1 // EmployeeManagement/Employee.java
2 package EmployeeManagement;
3
4 public class Employee {
5     private String name;
6     private int id;
7     private double salary;
8
9     // Constructor
10    public Employee(String name, int id, double salary) {
11        this.name = name;
12        this.id = id;
13        this.salary = salary;
14    }
15
16    // Getters (for controlled access to attributes)
17    public String getName() {
18        return name;
19    }
20
21    public int getId() {
22        return id;
23    }
24
25    public double getSalary() {
26        return salary;
27    }
28
29    // Method to display employee details
30    public void displayDetails() {
31        System.out.println("Employee Details:");
32        System.out.println("Name: " + name);
33        System.out.println("ID: " + id);
34        System.out.println("Salary: " + salary);
35    }
36 }
```

```
// MainProgram.java
import EmployeeManagement.Employee;

public class MainProgram {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // Create an Employee object
        Employee employee = new Employee(name:"Ayushi Vaswani", id:12345, salary:50000.00);

        // Display employee details
        employee.displayDetails();
    }
}
```

```
PS C:\Users\aaasth> & '
dt_ws\jdt.ls-java-proje
Employee Details:
Name: Ayushi vaswani
ID: 12345
Salary: 70000.0
```

2. In a Banking System, different types of users, such as bank employees and customers, need access to account operations with varying permissions. Create a package named `BankingSystem` containing a class `BankAccount` that has four methods with different access modifiers: a public method `deposit()` for adding money, a protected method `withdraw()` for authorized transactions, a default-access method `checkBalance()` to be accessed within the same package, and a private method `calculateInterest()` to compute interest internally. Then, create another package named `UserAccount` with a class `Customer` that tries to access these methods from an instance of `BankAccount`. Demonstrate which methods are accessible and explain why some cannot be accessed based on Java's access modifier rules.

```

1  // BankingSystem/BankAccount.java
2  package BankingSystem;
3  public class BankAccount {
4      private double balance;
5      // Constructor
6      public BankAccount(double initialBalance) {
7          this.balance = initialBalance;
8      }
9      // Public method for deposit
10     public void deposit(double amount) {
11         balance += amount;
12         System.out.println("Deposited: " + amount);
13     }
14     // Protected method for withdrawal (authorized transactions)
15     protected void withdraw(double amount) {
16         if (balance >= amount) {
17             balance -= amount;
18             System.out.println("Withdrawn: " + amount);
19         } else {
20             System.out.println(x:"Insufficient funds.");
21         }
22     }
23
24     // Default-access method for checking balance (within the same package)
25     void checkBalance() {
26         System.out.println("Current Balance: " + balance);
27         System.out.println("Interest: " + calculateInterest());
28     }
29
30     // Private method for calculating interest
31     private double calculateInterest() {
32         // Simplified interest calculation for demonstration
33         return balance * 0.05;
34     }
35 }

```

```

1  // UserAccount/Customer.java
2  package UserAccount;
3
4  import BankingSystem.BankAccount;
5
6  public class Customer {
7      Run | Debug | Run main | Debug main
8      public static void main(String[] args) {
9          BankAccount account = new BankAccount(initialBalance:1000.00);
10
11         // Accessing public method
12         account.deposit(amount:500.00);
13     }
14 }

```

```

• PS C:\Users\aaasth>
  dt_ws\jdt.ls-java-
  Deposited: 500.0

```

3. In a Student Records Management System, personal and academic details of students need to be stored securely while providing controlled access to their data. Create a package named `StudentRecords` containing a class `Student` with private attributes `studentId`, `name`, and `grade`. Provide a public constructor to initialize these attributes and a method `displayStudentInfo()` to print student details. Then, write another program outside the package that imports `StudentRecords`, creates `Student` objects, and displays their details. This exercise will illustrate how encapsulation is maintained in Java, allowing data to be accessed only through well-defined public methods while preventing direct modification of private fields.

```

1  // StudentRecords/Student.java
2  package StudentRecords;
3
4  public class Student {
5      private int studentId;
6      private String name;
7      private String grade;
8
9      // Public constructor to initialize attributes
10     public Student(int studentId, String name, String grade) {
11         this.studentId = studentId;
12         this.name = name;
13         this.grade = grade;
14     }
15
16     // Public method to display student information
17     public void displayStudentInfo() {
18         System.out.println(x:"Student Details:");
19         System.out.println("Student ID: " + studentId);
20         System.out.println("Name: " + name);
21         System.out.println("Grade: " + grade);
22     }
23 }

```

```

1  // MainProgram.java
2  import StudentRecords.Student;
3
4  public class studentdetail {
5      Run | Debug | Run main | Debug main
6      public static void main(String[] args) {
7          // Create Student objects
8          Student student1 = new Student(studentId:101, name:"Ayushi Vaswani", grade:"A");
9          Student student2 = new Student(studentId:102, name:"Disha Bhuvania", grade:"B");
10
11         // Display student information
12         student1.displayStudentInfo();
13         System.out.println(); // Add a line break
14         student2.displayStudentInfo();
15     }
16 }

```

```
PS C:\Users\aasth> & 'C:\Progr  
dt_ws\jdt.ls-java-project\bin'  
Student Details:  
Student ID: 101  
Name: Ayushi Vaswani  
Grade: A  
  
Student Details:  
Student ID: 102  
Name: Disha Bhuvania  
Grade: B
```

4. A security system needs to protect its authentication mechanism to prevent tampering. Implement a final class named `SecuritySystem` that includes a method `authenticateUser(String username, String password)`, which verifies user credentials. Then, attempt to create a subclass `AdvancedSecurity` that tries to extend `SecuritySystem` and override `authenticateUser()`. Observe the compilation error and explain why a final class cannot be inherited. In the `main()` method, create an instance of `SecuritySystem` and use the `authenticateUser()` method to validate login credentials. This will demonstrate how the final keyword enforces security by preventing unauthorized modifications to authentication logic.

```
// SecuritySystem.java  
public final class SecuritySystem {  
    private String[] validUsers = {"admin", "user1", "user2"};  
    private String[] validPasswords = {"password1", "password2", "password3"};  
  
    // Method to authenticate a user  
    public boolean authenticateUser(String username, String password) {  
        for (int i = 0; i < validUsers.length; i++) {  
            if (validUsers[i].equals(username) && validPasswords[i].equals(password)) {  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

```
public class Securities {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        SecuritySystem security = new SecuritySystem();  
  
        // Attempt to authenticate a user  
        boolean isAuthenticated = security.authenticateUser(username:"admin", password:"password1");  
  
        if (isAuthenticated) {  
            System.out.println(x:"Authentication successful!");  
        } else {  
            System.out.println(x:"Authentication failed.");  
        }  
    }  
}
```

```
PS C:\Users\aasth> & 'C:\  
dt_ws\jdt.ls-java-project\  
Authentication successful!
```